

C++ Function Parameters

The Depth of Data Transfer

Comprehensive Guide to Communication Between Functions

SYNTAX • MEMORY • PRACTICALS

1. Parameters vs. Arguments

While the terms are often used interchangeably, there is a technical distinction that is vital for understanding documentation and compiler errors.

Parameters (The Blueprint)

The variables listed in the function definition. They act as placeholders.

```
void greet(string name) { // 'name' is the parameter
    cout << "Hello " << name;
}
```

Arguments (The Reality)

The actual values you send to the function when you call it.

```
greet("CodeHive"); // "CodeHive" is the argument
```

Analogy: A **Parameter** is like the fields on an empty Job Application form (Name, Age). The **Argument** is the actual information you write in those fields when you submit it.

2. Multiple Parameters

Functions often need more than one piece of information to perform a task. You can list multiple parameters of different types, separated by commas.

```
void displayStudent(string name, int age, float gpa) {  
    cout << "Name: " << name << ", Age: " << age << ", GPA: " << gpa;  
}
```

The Rule of Order:

When calling a function with multiple parameters, the **order of arguments** must exactly match the **order of parameters**.

```
// Right:  
displayStudent("Alice", 20, 3.8);  
  
// Wrong:  
displayStudent(20, 3.8, "Alice"); // Compiler Error!
```

3. Default Parameters (Optional Values)

C++ allows you to provide a default value for a parameter. If the caller provides an argument, C++ uses that. If the caller leaves it blank, C++ uses the default.

```
void setVolume(int level = 50) {  
    cout << "Volume set to: " << level;  
}
```

Usage Scenarios:

```
setVolume(80); // Output: Volume set to: 80  
setVolume();   // Output: Volume set to: 50
```

Restriction: Default parameters must always be at the **end** of the parameter list. You cannot have a default parameter followed by a non-default one.

```
void test(int a = 10, int b) // ❌ INVALID
```

4. Return Values

The return keyword is a function's way of talking back to the main program. It terminates the function and provides a result.

```
int multiply(int a, int b) {  
    return a * b; // 'int' matches the return type  
}
```

Capturing the Value:

```
int result = multiply(10, 5);  
// The value 50 is physically moved into the 'result' variable.
```

Pro Tip: A function can only return **one** value. To return multiple values, you would need to use Structures or Pointers (covered in later pages).

5. Pass by Value (The Clone)

By default, C++ uses "Pass by Value." This means the function receives a **copy** of the data. The original data in the main program is protected.

```
void increment(int x) {  
    x = x + 1;  
}  
  
int main() {  
    int num = 5;  
    increment(num);  
    cout << num; // Still outputs 5!  
}
```

Why it happens:

The variable `x` inside the function exists in a different memory location than `num`. Modifying `x` is like editing a photocopy of a document; the original remains unchanged.

6. Pass by Reference (The Original)

If you want a function to be able to modify the original variable, you use the **ampersand (&)** operator in the parameter definition.

```
void realIncrement(int &x) {  
    x = x + 1;  
}  
  
int main() {  
    int num = 5;  
    realIncrement(num);  
    cout << num; // Now outputs 6!  
}
```

Key Advantages:

- **Efficiency:** No time or memory is wasted copying large data sets.
- **Persistence:** Changes last even after the function finishes.

7. Comparison: Value vs. Reference

Feature	Pass by Value	Pass by Reference
Data Impact	Original stays same	Original can be changed
Memory	Uses extra memory (Copy)	Uses original memory (Alias)
Speed	Slower for large objects	Very fast
Safety	High (No accidental changes)	Lower (Risks changing data)

Best Practice: Use `const` with references (`const int &x`) if you want the speed of a reference but want to prevent the function from changing the data.

8. Passing Arrays to Functions

Arrays are unique. Unlike single variables, arrays are passed by reference **automatically**. You do not need the & symbol.

```
void printArray(int myArr[], int length) {
    for(int i = 0; i < length; i++) {
        cout << myArr[i] << " ";
    }
}
```

Crucial Requirement: The Size

When you pass an array, the function loses track of how many items are in it (this is called "Array Decay"). You must **always** pass a second parameter representing the size.

```
int main() {
    int scores[3] = {90, 85, 92};
    printArray(scores, 3);
}
```

9. Passing Structures

Custom data types (structs) can be passed just like basic types. However, because structs can be very large, they are almost always passed by **Reference** in professional code.

```
struct Hero {  
    string name;  
    int health;  
};  
  
void restoreHealth(Hero &h) {  
    h.health = 100;  
}
```

Call Logic:

```
Hero player1 = {"Warrior", 45};  
restoreHealth(player1);  
// player1.health is now 100
```

10. Overloading Parameters

Overloading allows one function name to handle different types of parameters. This is determined by the **Parameter List**.

```
// Version 1: Integers
int sum(int x, int y) { return x + y; }

// Version 2: Doubles
double sum(double x, double y) { return x + y; }
```

The compiler checks the arguments you pass. If you pass `5.5`, it ignores Version 1 and executes Version 2 automatically.

11. Practical Implementation

Let's combine multiple parameter concepts into a single "Shopping Cart" logic example.

```
double calculateTotal(double price, int qty, double tax = 0.08) {
    double subtotal = price * qty;
    return subtotal + (subtotal * tax);
}

int main() {
    // Normal purchase
    cout << "Standard: $" << calculateTotal(10.0, 2) << endl;

    // Purchase with custom tax (e.g., 0.12)
    cout << "Luxury: $" << calculateTotal(500.0, 1, 0.12) << endl;
}
```

Notice how we utilized **Return Values**, **Multiple Parameters**, and **Default Parameters** in one efficient block.

12. Advanced Content: The Const Ref

In high-performance C++, you will often see `(const string &text)`. This is the ultimate "Professional" parameter style.

1. **The & :** Ensures the function doesn't make a slow copy of a long string.
2. **The const :** Ensures the function can't accidentally delete or change the string.

```
void displayBigText(const string &msg) {  
    // Fast and Safe!  
    cout << msg;  
}
```

13. Troubleshooting & Analysis

The "Missing Size" Error

Passing an array without a size variable. The function will read random items in memory after the array ends.

Dangling References

Trying to return a reference to a variable that was created inside the function. Once the function ends, the reference points to a dead variable.

Checklist for Parameter Success:

- Does my return type match the value being returned?
- Are my default parameters at the end of the list?
- Am I passing small numbers by value and large objects by reference?

14. Summary & Final Lab

Functions are only as good as the data they receive. By mastering parameters, you control the flow of energy through your program.

```
// FINAL LAB CHALLENGE:  
// 1. Write a function with 2 ints as parameters.  
// 2. Pass them by reference.  
// 3. Swap their values.  
// 4. Verify in Main that they have swapped.
```

Quick Summary:

- **Parameters** are the names; **Arguments** are the values.
- **Default** values make parameters optional.
- **Reference (&)** allows modifications to the original variable.
- **Arrays** decay to pointers; always pass their size.
- **Const** ensures safety when using references.

MODULE COMPLETE: FUNCTION PARAMETERS MASTERCLASS